

Assignment – Backend Developer

Envobyte Ltd. — Technical Assignment

Tech Stack: PHP / Laravel · MySQL 8.0+ · Laravel Queues

Expected Effort: Approximately 10 hours

Existing Project Work: Monica CRM

Background

You will work with **Monica**, an open-source personal CRM built with Laravel. Monica helps users manage contacts, activities, reminders, notes, journal entries, and calls.

Project URL: <https://github.com/monicahq/monica>

This assignment evaluates your ability to understand an unfamiliar codebase, implement a reliable background process, maintain data consistency, and write properly tested Laravel code.

Setup

1. Fork the Monica repository.
 2. Set it up locally following the project documentation.
 3. Create a branch named `envobyte-assignment`.
-

Project: Background Contact Import System

Monica allows users to import contacts from CSV files. Processing a large file during an HTTP request can cause slow responses, memory issues, or request timeouts. Users also need visibility into the progress and result of an import.

Your task is to redesign the relevant CSV import flow so that the file is processed in the background, progress is recorded, and an invalid row does not stop the entire import.

Problem Analysis

Before implementation, inspect Monica's existing import flow, including the relevant:

- Controller or API endpoint
- Import service, parser, action, or job
- Contact-creation and validation logic
- Database models and authorization conventions

In the README, briefly explain:

- How the existing import flow works
- Which existing components you reused or modified
- Any important assumptions you made

Requirements

1. Import Initiation

Implement or update the following endpoint:

Method	Path	Description
POST	/api/import	Upload a CSV file and begin background processing

The endpoint must:

- Require an authenticated user.
- Validate that a supported CSV file was uploaded.
- Store the file in a non-public location.
- Create an import record with **pending** status.
- Dispatch a Laravel queued job.
- Return the created import information without processing the contacts inside the HTTP request.

Example Response

Response: 201

```
{
  "data": {
    "id": 1,
    "filename": "contacts.csv",
    "total_rows": 0,
    "processed_rows": 0,
    "failed_rows": 0,
```

```
"status": "pending",
"created_at": "2026-06-03T12:00:00Z"
}
}
```

2. Import Data Model

Create an `import_jobs` table or an equivalent structure that follows Monica's existing naming conventions.

The import record should contain at least:

Column	Purpose
<code>id</code>	Primary key
<code>account_id</code>	Account ownership and scoping
<code>user_id</code>	User who initiated the import
<code>filename</code>	Original uploaded filename
<code>file_path</code>	Stored file location
<code>total_rows</code>	Total number of data rows
<code>processed_rows</code>	Number of rows already attempted
<code>failed_rows</code>	Number of rejected rows
<code>status</code>	<code>pending</code> , <code>processing</code> , <code>completed</code> , or <code>failed</code>
<code>failure_message</code>	System-level failure details, when applicable
<code>started_at</code>	When background processing began
<code>completed_at</code>	When processing ended

You may add or change columns where necessary. Briefly explain any significant decisions in the README.

3. Background Processing

Implement a Laravel queued job that:

- Changes the import status from `pending` to `processing`.
- Reads the CSV incrementally without loading the complete file into memory.

- Processes rows in chunks of approximately 50 rows.
- Creates contacts through Monica's existing contact-creation flow where practical.
- Updates `processed_rows` and `failed_rows` after each chunk.
- Sets the status to `completed` after all rows have been handled.
- Sets the status to `failed` and records a meaningful message if a system-level error prevents completion.

Using a single queued job that processes the file in chunks is sufficient. Separate jobs for individual chunks are not required.

4. Per-Row Error Isolation

An invalid CSV row must not stop the complete import.

Your implementation must:

- Validate each row independently.
- Continue processing the remaining rows when one row fails.
- Record the row number and validation error.
- Increment `failed_rows` for every rejected row.
- Keep the import status as `completed` when processing finishes, even if some rows were rejected.
- Set the overall status to `failed` if no contact is imported successfully.

You may store row-level errors in an `import_errors` table or another appropriate structure. Briefly justify your decision.

Example errors may include:

- Missing required name
- Invalid email format
- Invalid or unsupported field value
- Failure during contact creation

5. Progress Tracking

Implement the following endpoint:

Method	Path	Description
GET	<code>/api/import/:id</code>	Return the current import status and progress

The endpoint must:

- Enforce authentication and account ownership.
- Return data only for an import accessible to the authenticated user.
- Calculate progress using the import record.
- Not count or query imported contact rows to determine progress.

Example Response

Response: 200

```
{
  "data": {
    "id": 1,
    "filename": "contacts.csv",
    "total_rows": 500,
    "processed_rows": 320,
    "failed_rows": 2,
    "status": "processing",
    "progress_pct": 64,
    "started_at": "2026-06-03T12:00:05Z",
    "completed_at": null
  }
}
```

Use Monica's existing API response and authorization conventions where practical.

6. Basic Retry Safety

Laravel may retry a queued job after a timeout or unexpected failure. The import job must be safe to retry without creating duplicate contacts for successfully committed rows. Explain the transaction or idempotency strategy and include one automated test covering retry behavior.

In the README, explain:

- What could happen if the job crashes after creating a contact but before updating the progress counter
 - How your solution reduces or prevents duplicate processing
 - Any remaining limitations in your approach
-

Tests Required

Include automated tests covering the following areas:

1. Import Initiation

Verify that:

- The uploaded file is validated.
- An import record is created.
- The queued job is dispatched.
- Contacts are not processed inside the HTTP request.

2. Background Processing and Progress

Verify that:

- Valid rows are imported.
- Progress counts are updated.
- The import reaches the correct final status.

3. Per-Row Error Isolation

Verify that:

- An invalid row is recorded as failed.
- Valid rows continue to be processed.
- The final processed and failed counts are correct.

Additional requirements:

- Tests must run using a single documented command, such as `php artisan test`.
- The README must contain all necessary test instructions.
- Submissions without runnable automated tests may be considered incomplete.

4. Retry and Duplicate Protection

Verify that:

- A job can be retried after partial processing.
- Successfully committed rows are not imported twice.
- Progress counts remain consistent after the retry.

Technical Questions

Answer the following questions briefly in the README. Implementation is not required.

1. How would you detect that an import has remained in `processing` for an unusually long time?
2. How would you allow a user to cancel a running import?
3. How would you handle two uploads of the same file?
4. What metrics would you monitor for this import system?

Keep the answers concise and focused on practical production considerations.

Optional Bonus Features

The following features are optional and are not required for full marks:

- Import cancellation
- Downloadable CSV containing rejected rows and their errors
- Duplicate-file detection using a file hash
- Additional retry or idempotency protection
- Additional meaningful automated tests

Bonus features should not reduce the quality or completeness of the mandatory requirements.

What We Are Evaluating

Area	Weight
Understanding and integration with the existing codebase	15%
Queue job and memory-efficient processing	25%
Database design and progress tracking	15%
Validation and per-row error isolation	15%
Code quality and Laravel conventions	10%
Automated-test quality	15%
README and technical reasoning	5%

Strong Candidates Will

- Understand the relevant existing code before implementing changes.
 - Follow Monica's architecture and Laravel conventions.
 - Keep controllers thin and responsibilities properly separated.
 - Handle validation and failures without breaking the complete import.
 - Write tests that verify meaningful behavior.
 - Clearly explain assumptions, trade-offs, and retry limitations.
 - Produce readable Git history with logical commits.
-

Technical Review

Shortlisted candidates may be invited to a brief technical review. You should be prepared to:

- Explain the import flow and architecture.
 - Explain how progress and row-level failures are tracked.
 - Discuss what happens when the queued job fails or is retried.
 - Make or explain a small change to the submitted implementation.
-

Submission Requirements

Submit a public GitHub repository containing:

- All required migrations and application code
- Automated tests
- A [README.md](#) containing:
 - Setup instructions
 - Existing-flow analysis
 - Implementation approach
 - Assumptions and limitations
 - Retry-safety explanation
 - Answers to the technical questions
 - Test instructions

Submit the repository link through the provided assignment submission form and keep it public.